

Exercise 2: Project Genesis: /init and the CLAUDE.md Constitution

DEV CYCLE	Design (project setup)
CC FEATURES	/init, CLAUDE.md memory file, @file imports, /memory
SPEC TRACES	NFR-01 (stack), POLICY.md (guardrails)
TIME	25 min
YOU PRODUCE	A Python venv, a hand-tuned CLAUDE.md that points at docs/SPEC.md, and proof that Claude obeys it

Objective

Give Claude its standing instructions. CLAUDE.md is the agent's constitution for this repository: it is loaded at the start of every session and anchors your stack, conventions, and the things Claude must not touch. In a spec-driven project, CLAUDE.md does one more job: it tells Claude where the contract lives (`docs/SPEC.md`) and that the contract beats improvisation. A good CLAUDE.md is the highest-leverage file you will write all day.

Walk-through

1 Create the environment

Exit Claude Code (Ctrl+C twice or `/exit`). Create a virtual environment inside the cloned repo and install the pinned stack yourself, so you know the ground truth before Claude touches anything. The only platform difference is how you create and activate the venv.

WINDOWS · POWERSHELL

```
PS> python -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
PS> pip install -r requirements.txt
PS> Copy-Item .env.example .env
```

MACOS / LINUX · BASH OR ZSH

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
$ cp .env.example .env
```

Open `.env` in VS Code and paste your real Anthropic key into `ANTHROPIC_API_KEY`. Set `DISCHARGE_API_KEY` to any long random string. You will not need either until Exercise 5, but set them now while you are here.

VALIDATE Your prompt shows `(.venv). pip list` includes `fastapi`, `sqlalchemy`, `anthropic`, `streamlit` and `python-multipart`. `git status` does NOT list `.env` (it is git-ignored). If it does, stop and fix `.gitignore` before anything else.

SCREENSHOT SLOT *pip install output and git status showing .env is ignored*

2 Launch Claude Code and run /init

Start Claude Code from the repo root, then run `/init`. Claude inspects the directory and drafts a CLAUDE.md. Because this repo already contains a brief and a spec, watch how much of the draft comes from reading them. Review what it proposes rather than accepting blindly.

CLAUDE.md serves two purposes: it guides Claude through the codebase (commands, architecture, style), and it carries your specific, binding directions.

RUN IN TERMINAL

```
PS> claude
```

INSIDE THE CLAUDE CODE SESSION

```
> /init
```

VALIDATE A CLAUDE.md appears in the repo root. Open it: it should mention FastAPI, SQLite, the Anthropic SDK, and ideally docs/SPEC.md. Note anything it got wrong or left vague: the generic draft rarely captures the dose rule or the four-eyes rule.

SCREENSHOT SLOT /init output and the generated CLAUDE.md open in VS Code

3 Rewrite CLAUDE.md to be specific

Replace the generated body with a precise constitution. The @docs/SPEC.md and @docs/POLICY.md lines are imports: Claude loads those files into context along with CLAUDE.md, so the spec is always in view. Keep CLAUDE.md itself short and let it point at the detail.

CREATE / PASTE → CLAUDE.MD

```
# DischargeFlow

## What this is
A clinician-in-the-loop discharge system. PoC, not production, not a medical device.
Synthetic data only. Journey: clinician submits a structured discharge record ->
Claude drafts a plain-language patient summary grounded in reference summaries (RAG) ->
a DIFFERENT clinician approves or rejects -> only an approved summary is released.

## Source of truth
- @docs/SPEC.md is the contract (tables, FR/AI/SEC requirement IDs, API). Build to it.
- @docs/POLICY.md holds the safety, PHI and security rules.
- If a request conflicts with the spec, stop and ask. Propose a spec change instead.
- Put requirement IDs in docstrings, test names and commit messages (e.g. FR-10).

## Stack
- Python 3.12, FastAPI, Uvicorn, SQLAlchemy 2.x on SQLite (dischargeflow.db)
- pydantic v2, httpx, python-multipart, pytest, anthropic SDK, Streamlit

## Conventions
- Layout: app/main.py, db.py, models.py, schemas.py, routes.py (+ modules per SPEC)
- Type hints on every function. Docstrings on every endpoint and module.
- Doses are fixed-point integers dose_x1000 plus a dose_unit. Never floats.
- The model never writes doses: the medication table is rendered from the DB.
- Datetimes are UTC. Secrets come from environment variables via python-dotenv.

## Safety rules
- Mask PHI (name, MRN, DOB, phone, email, IDs) before any external API call.
- Never log PHI. Use patient_ref in logs, audit events and errors.
- Four-eyes: reviewer_id must differ from submitted_by.
- Tests always mock the Anthropic client and run offline.

## Run and test
- Run: python -m uvicorn app.main:app --reload
- Test: python -m pytest -q

## Do not touch
- Do not edit .venv, .git, .env or dischargeflow.db directly.
- Do not add third-party dependencies without asking me first.
```

- Do not invent endpoints, tables or fields that are not in docs/SPEC.md.

VALIDATE Save CLAUDE.md. Three lines will pay off most today: doses as integer `dose_x1000`, "the model never writes doses", and "do not invent endpoints that are not in the spec". They pre-empt the most common AI mistakes in this domain.

4 Prove Claude is reading it

Start a fresh session (`/clear`) so nothing from this conversation leaks in, then ask questions whose answers exist only in your constitution and the spec it imports.

TYPE THIS PROMPT INTO CLAUDE CODE

Without writing any code, answer from CLAUDE.md and the files it imports only: (1) how is a 2.5 mg dose stored? (2) who is allowed to approve a draft? (3) what must happen to the patient's name before we call the Claude API?

VALIDATE Claude answers: `dose_x1000 = 2500` with `dose_unit = mg`; a reviewing clinician who is not the submitting clinician (FR-10); the name is masked to `[PATIENT]` before egress (SEC-03). If it guesses, check that CLAUDE.md is in the repo root and the `@` paths are spelled correctly.

SCREENSHOT SLOT *Claude answering the three questions from CLAUDE.md*

5 Add a memory mid-session

You will learn lessons during the build that belong in the constitution. Two quick ways to record one: run `/memory` to open the memory file in your editor, or simply tell Claude to add the rule to CLAUDE.md and approve the edit. Add one now.

TYPE THIS PROMPT INTO CLAUDE CODE

Add this rule to the Conventions section of CLAUDE.md: "Every new endpoint docstring must cite the SPEC requirement ID it implements."

INSIDE THE CLAUDE CODE SESSION

```
> /memory
```

VALIDATE CLAUDE.md now contains the new rule. `/memory` shows which memory files are loaded (project CLAUDE.md, plus your user-level file if you have one).

SPEC LINK From now on, every prompt you write should name the requirement ID it is building (for example "implement FR-04"). That is what "spec-driven" means in practice: the spec, not the chat, is the memory of what was agreed.

What success looks like

- CLAUDE.md describes DischargeFlow precisely and imports docs/SPEC.md and docs/POLICY.md.
- A fresh session answers the dose, four-eyes and PHI questions correctly from CLAUDE.md alone.
- You know two ways to update the constitution: edit the file directly, or ask Claude and approve the change; `/memory` shows what is loaded.
- `.env` holds your key and is provably git-ignored.

Common pitfalls (Windows and macOS)

- If `.\.venv\Scripts\Activate.ps1` is blocked on Windows, set the per-session execution policy as in Exercise 1, then re-run it.
- CLAUDE.md only applies to the folder tree it sits in. Start `cclaude` from the repo root.

- Resist making CLAUDE.md enormous. Tight rules in CLAUDE.md, detail in SPEC.md via @import.
- Never paste your API key into CLAUDE.md or any prompt. It lives in .env only.

Go deeper • AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- Add a permissions block to .claude/settings.json that allows Bash(`python -m pytest:*`) without prompting and denies Read(`./.env`). Prove both rules with one prompt each.
- Measure the token cost of your memory: note `/context` before and after adding the `@docs/SPEC.md` import. Decide what belongs inline in CLAUDE.md and what should stay behind an import.

Stretch goal

Add a user-level memory at `~/ .claude/CLAUDE.md` with your personal defaults (for example, "explain trade-offs before large refactors; prefer small diffs"). Project and user memory stack together; confirm both appear in `/memory`.